

Stacks and Queues

Abstract Data Types: Stacks and Queues

Stacks and queues are abstract data types (ADTs) in computer science, each with a unique set of operations and principles.

Stacks

A stack operates on the principle of “Last In, First Out” (LIFO). The last element added to the stack will be the first one to be removed.

Key Operations:

1. **Push:** Add an element to the top of the stack.
2. **Pop:** Remove and return the top element of the stack.
3. **Peek:** Return the top element without removing it.
4. **IsEmpty:** Check if the stack is empty.

Queues

A queue operates on the principle of “First In, First Out” (FIFO). The first element added will be the first one to be removed.

Key Operations:

1. **Enqueue:** Add an element to the end of the queue.
2. **Dequeue:** Remove and return the front element of the queue.
3. **Front:** Return the front element without removing it.
4. **IsEmpty:** Check if the queue is empty.

Implementation Techniques

Python Example: Stack Implementation

A stack can be implemented using a list in Python:

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        return None
```

```

def peek(self):
    if not self.is_empty():
        return self.items[-1]
    return None

def is_empty(self):
    return len(self.items) == 0

```

Python Example: Queue Implementation

A queue can also be implemented using a list:

```

class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.insert(0, item)

    def dequeue(self):
        if not self.is_empty():
            return self.items.pop()
        return None

    def front(self):
        if not self.is_empty():
            return self.items[-1]
        return None

    def is_empty(self):
        return len(self.items) == 0

```

Real-world Applications

Stacks

1. **Undo Mechanisms:** In text editors, undo operations are implemented using stacks.
2. **Expression Evaluation:** Used in algorithmic expression evaluation such as postfix/prefix calculations.
3. **Function Calls:** Program execution stacks store information about function calls.

Queues

1. **Task Scheduling:** In operating systems, queues manage processes for task scheduling.
2. **Data Buffering:** Queues are used for buffering data streams.
3. **Asynchronous Data Transfer:** Queues facilitate asynchronous transfer of data between different parts of an application, like in event handling systems.

Advanced Example: Implementing a Queue Using Two Stacks

This example demonstrates the versatility of these data structures:

```
class QueueUsingStacks:
    def __init__(self):
        self.in_stack = Stack()
        self.out_stack = Stack()

    def enqueue(self, item):
        self.in_stack.push(item)

    def dequeue(self):
        if self.out_stack.is_empty():
            while not self.in_stack.is_empty():
                self.out_stack.push(self.in_stack.pop())
        return self.out_stack.pop()
```

In this implementation, we use two stacks (`in_stack` and `out_stack`) to mimic the behavior of a queue. This illustrates how stacks can be utilized to form other complex data structures like queues.